

第50章 读写内部 FLASH

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库说明文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

50.1 STM32 的内部 FLASH 简介

在 STM32 芯片内部有一个 FLASH 存储器，它主要用于存储代码，我们在电脑上编写好应用程序后，使用下载器把编译后的代码文件烧录到该内部 FLASH 中，由于 FLASH 存储器的内容在掉电后不会丢失，芯片重新上电复位后，内核可从内部 FLASH 中加载代码并运行，见图 50-1。

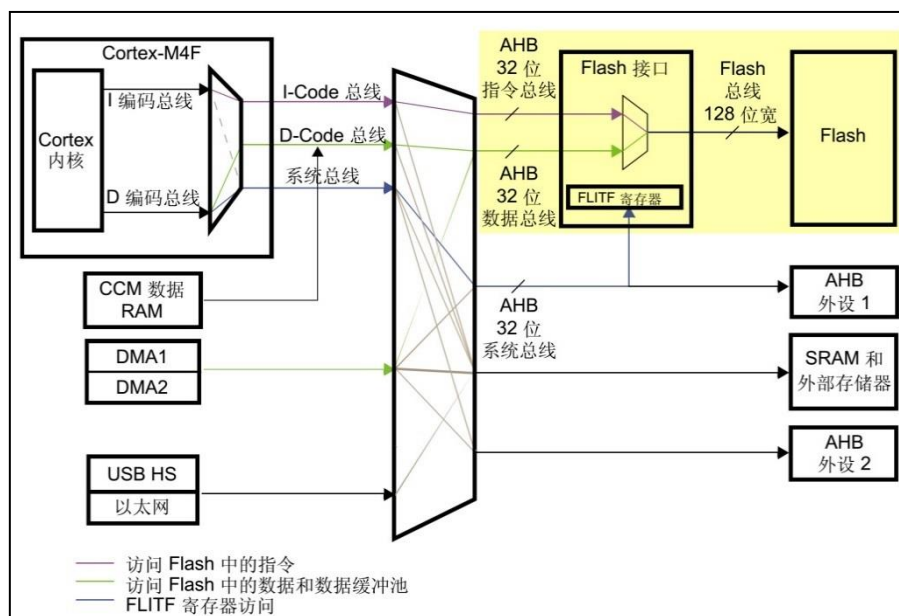


图 50-1 STM32 的内部框架图

除了使用外部的工具（如下载器）读写内部 FLASH 外，STM32 芯片在运行的时候，也能对自身的内部 FLASH 进行读写，因此，若内部 FLASH 存储了应用程序后还有剩余的空间，我们可以把它像外部 SPI-FLASH 那样利用起来，存储一些程序运行时产生的需要掉电保存的数据。

由于访问内部 FLASH 的速度要比外部的 SPI-FLASH 快得多，所以在紧急状态下常常会使用内部 FLASH 存储关键记录；为了防止应用程序被抄袭，有的应用会禁止读写内部 FLASH 中的内容，或者在第一次运行时计算加密信息并记录到某些区域，然后删除自身的部分加密代码，这些应用都涉及到内部 FLASH 的操作。

1. 内部 FLASH 的构成

STM32 的内部 FLASH 包含主存储器、系统存储器、OTP 区域以及选项字节区域，它们的地址分布及大小见表 50-1。

表 50-1 STM32 内部 FLASH 的构成

区域	块	名称	块地址	大小
----	---	----	-----	----

主存储器	块 1	扇区 0	0x0800 0000 - 0x0800 3FFF	16 Kbytes
		扇区 1	0x0800 4000 - 0x0800 7FFF	16 Kbytes
		扇区 2	0x0800 8000 - 0x0800 BFFF	16 Kbytes
		扇区 3	0x0800 C000 - 0x0800 FFFF	16 Kbyte
		扇区 4	0x0801 0000 - 0x0801 FFFF	64 Kbytes
		扇区 5	0x0802 0000 - 0x0803 FFFF	128 Kbytes
		扇区 6	0x0804 0000 - 0x0805 FFFF	128 Kbytes
		扇区 7	0x0806 0000 - 0x0807 FFFF	128 Kbytes
		扇区 8	0x0808 0000 - 0x0809 FFFF	128 Kbytes
		扇区 9	0x080A 0000 - 0x080B FFFF	128 Kbytes
		扇区 10	0x080C 0000 - 0x080D FFFF	128 Kbytes
		扇区 11	0x080E 0000 - 0x080F FFFF	128 Kbytes
	块 2	扇区 12	0x0810 0000 - 0x0810 3FFF	16 Kbytes
		扇区 13	0x0810 4000 - 0x0810 7FFF	16 Kbytes
		扇区 14	0x0810 8000 - 0x0810 BFFF	16 Kbytes
		扇区 15	0x0810 C000 - 0x0810 FFFF	16 Kbyte
		扇区 16	0x0811 0000 - 0x0811 FFFF	64 Kbytes
		扇区 17	0x0812 0000 - 0x0813 FFFF	128 Kbytes
		扇区 18	0x0814 0000 - 0x0815 FFFF	128 Kbytes
		扇区 19	0x0816 0000 - 0x0817 FFFF	128 Kbytes
		扇区 20	0x0818 0000 - 0x0819 FFFF	128 Kbytes
		扇区 21	0x081A 0000 - 0x081B FFFF	128 Kbytes
		扇区 22	0x081C 0000 - 0x081D FFFF	128 Kbytes
		扇区 23	0x081E 0000 - 0x081F FFFF	128 Kbytes
系统存储区			0x1FFF 0000 - 0x1FFF 77FF	30 Kbytes
OTP 区域			0x1FFF 7800 - 0x1FFF 7A0F	528 bytes
选项字节	块 1	0x1FFF C000 - 0x1FFF C00F		16 bytes
	块 2	0x1FFE C000 - 0x1FFE C00F		16 bytes

各个存储区域的说明如下：

❑ 主存储器

一般我们说 STM32 内部 FLASH 的时候，都是指这个主存储器区域，它是存储用户应用程序的空间，芯片型号说明中的 1M FLASH、2M FLASH 都是指这个区域的大小。主存储器分为两块，共 2MB，每块内分 12 个扇区，其中包含 4 个 16KB 扇区、1 个 64KB 扇区和 7 个 128KB 的扇区。如我们实验板中使用的 STM32F407ZGT6 型号芯片，它的主存储区域大小为 1MB，所以它只包含有表中的扇区 0-扇区 11。

与其它 FLASH 一样，在写入数据前，要先按扇区擦除，而有的时候我们希望能以小规格操纵存储单元，所以 STM32F42x/43x 针对 1MB FLASH 的产品还提供了—种双块的存储格式，见表 50-2。(2M 的产品按表 50-1 的格式)

表 50-2 1MB 产品的双块存储格式

1M 字节单块存储器的扇区分配(默认)			1M 字节双块存储器的扇区分配		
DB1M=0			DB1M=1		
主存储器	扇区号	扇区大小	主存储器	扇区号	扇区大小
1MB	扇区 0	16 Kbytes	Bank 1 512KB	扇区 0	16 Kbytes
	扇区 1	16 Kbytes		扇区 1	16 Kbytes

	扇区 2	16 Kbytes		扇区 2	16 Kbytes
	扇区 3	16 Kbytes		扇区 3	16 Kbytes
	扇区 4	64 Kbytes		扇区 4	64 Kbytes
	扇区 5	128 Kbytes		扇区 5	128 Kbytes
	扇区 6	128 Kbytes		扇区 6	128 Kbytes
	扇区 7	128 Kbytes		扇区 7	128 Kbytes
	扇区 8	128 Kbytes		扇区 12	16 Kbytes
	扇区 9	128 Kbytes	Bank 2 512KB	扇区 13	16 Kbytes
	扇区 10	128 Kbytes		扇区 14	16 Kbytes
	扇区 11	128 Kbytes		扇区 15	16 Kbytes
	-	-		扇区 16	64 Kbytes
	-	-		扇区 17	128 Kbytes
	-	-		扇区 18	128 Kbytes
	-	-		扇区 19	128 Kbytes

通过配置 FLASH 选项控制寄存器 FLASH_OPTCR 的 DB1M 位，可以切换这两种格式，切换到双块模式后，扇区 8-11 的空间被转移到扇区 12-19 中，扇区细分了，总容量不变。

要强调的是：本实验板采用的是 STM32F40x 系列的芯片，它没有双块存储格式，也不存在扇区 12-23，仅 STM32F42x/43x 系列产品才支持扇区 12-23。

❑ 系统存储区

系统存储区是用户不能访问的区域，它在芯片出厂时已经固化了启动代码，它负责实现串口、USB 以及 CAN 等 ISP 烧录功能。

❑ OTP 区域

OTP(One Time Program)，指的是只能写入一次的存储区域，容量为 512 字节，写入后数据就无法再更改，OTP 常用于存储应用程序的加密密钥。

❑ 选项字节

选项字节用于配置 FLASH 的读写保护、电源管理中的 BOR 级别、软件/硬件看门狗等功能，这部分共 32 字节。可以通过修改 FLASH 的选项控制寄存器修改。

50.2 对内部 FLASH 的写入过程

1. 解锁

由于内部 FLASH 空间主要存储的是应用程序，是非常关键的数据，为了防止误操作修改了这些内容，芯片复位后默认会结 FLASH 上锁，这个时候不允许设置 FLASH 的控制寄存器，并且不能对修改 FLASH 中的内容。

所以对 FLASH 写入数据前，需要先给它解锁。解锁的操作步骤如下：

- (1) 往 Flash 密钥寄存器 FLASH_KEYR 中写入 KEY1 = 0x45670123
- (2) 再往 Flash 密钥寄存器 FLASH_KEYR 中写入 KEY2 = 0xCDEF89AB

2. 数据操作位数

在内部 FLASH 进行擦除及写入操作时，电源电压会影响数据的最大操作位数，该电源电压可通过配置 FLASH_CR 寄存器中的 PSIZE 位改变，见表 50-3。

表 50-3 数据操作位数

电压范围	2.7 - 3.6 V (使用外部 Vpp)	2.7 - 3.6 V	2.1 - 2.7 V	1.8 - 2.1 V
位数	64	32	16	8
PSIZE(1:0)配置	11b	10b	01b	00b

最大操作位数会影响擦除和写入的速度，其中 64 位宽度的操作除了配置寄存器位外，还需要在 Vpp 引脚外加一个 8-9V 的电压源，且其供电时间不得超过一小时，否则 FLASH 可能损坏，所以 64 位宽度的操作一般是在量产时对 FLASH 写入应用程序时才使用，大部分应用场合都是用 32 位的宽度。

3. 擦除扇区

在写入新的数据前，需要先擦除存储区域，STM32 提供了扇区擦除指令和整个 FLASH 擦除(批量擦除)的指令，批量擦除指令仅针对主存储区。

扇区擦除的过程如下：

- (1) 检查 FLASH_SR 寄存器中的“忙碌寄存器位 BSY”，以确认当前未执行任何 Flash 操作；
- (2) 在 FLASH_CR 寄存器中，将“激活扇区擦除寄存器位 SER”置 1，并设置“扇区编号寄存器位 SNB”，选择要擦除的扇区；
- (3) 将 FLASH_CR 寄存器中的“开始擦除寄存器位 STRT”置 1，开始擦除；
- (4) 等待 BSY 位被清零时，表示擦除完成。

4. 写入数据

擦除完毕后即可写入数据，写入数据的过程并不是仅仅使用指针向地址赋值，赋值前还需要配置一系列的寄存器，步骤如下：

- (1) 检查 FLASH_SR 中的 BSY 位，以确认当前未执行任何其它的内部 Flash 操作；
- (2) 将 FLASH_CR 寄存器中的“激活编程寄存器位 PG”置 1；
- (3) 针对所需存储器地址（主存储器块或 OTP 区域内）执行数据写入操作；
- (4) 等待 BSY 位被清零时，表示写入完成。

50.3 查看工程的空间分布

由于内部 FLASH 本身存储有程序数据，若不是有意删除某段程序代码，一般不应修改程序空间的内容，所以在内部 FLASH 存储其它数据前需要了解哪一些空间已经写入了程序代码，存储了程序代码的扇区都不应作任何修改。通过查询应用程序编译时产生的“*.map”后缀文件，可以了解程序存储到了哪些区域，它在工程中的打开方式见图 50-2，也可以到工程目录中的“Listing”文件夹中找到。

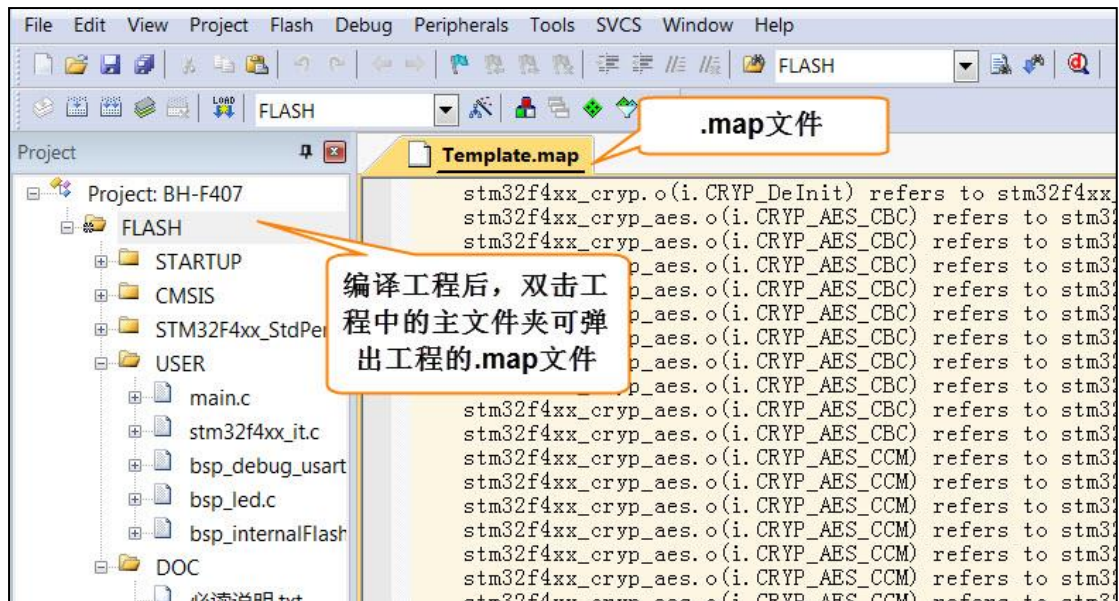


图 50-2 打开工程的.map 文件

打开 map 文件后，查看文件最后部分的区域，可以看到一段以“Memory Map of the image”开头的记录(若找不到可用查找功能定位)，见代码清单 50-1。

代码清单 50-1 map 文件中的存储映像分布说明

```
1 =====
2
3 Memory Map of the image //存储分布映像
4
5 Image Entry point : 0x08000189
6 /*程序 ROM 加载空间*/
7 Load Region LR_IROM1 (Base: 0x08000000, Size: 0x00000aa4, Max: 0x00100000, ABSOLUTE)
8 /*程序 ROM 执行空间*/
9 Execution Region ER_IROM1 (Base: 0x08000000, Size: 0x00000a90, Max: 0x00100000, ABSOLUTE)
10 /*地址分布列表*/
11 Base Addr      Size      Type    Attr    Idx    E Section Name      Object
12
13 0x08000000      0x00000188    Data    RO       3      RESET               startup_stm32f40xx.o
14 0x08000188      0x00000000    Code    RO      4963      *.ARM.Collect$$$$00000000    mc_w.l(entry.o)
15 0x08000188      0x00000004    Code    RO      5226      *.ARM.Collect$$$$00000001    mc_w.l(entry2.o)
16 0x0800018c      0x00000004    Code    RO      5229      *.ARM.Collect$$$$00000004    mc_w.l(entry5.o)
17 0x08000190      0x00000000    Code    RO      5231      *.ARM.Collect$$$$00000008    mc_w.l(entry7b.o)
18 0x08000190      0x00000000    Code    RO      5233      *.ARM.Collect$$$$0000000A    mc_w.l(entry8b.o)
19 0x08000190      0x00000008    Code    RO      5234      *.ARM.Collect$$$$0000000B    mc_w.l(entry9a.o)
20 0x08000198      0x00000000    Code    RO      5236      *.ARM.Collect$$$$0000000D    mc_w.l(entry10a.o)
21 /*...此处省略大部分内容*/
22 0x08000902      0x00000002    PAD
23 0x08000904      0x00000010    Code    RO      4967      i.__printf$bare             mc_w.l(printfb.o)
24 0x08000914      0x0000000e    Code    RO      5268      i.__scatterload_copy        mc_w.l(handlers.o)
25 0x08000922      0x00000002    Code    RO      5269      i.__scatterload_null         mc_w.l(handlers.o)
26 0x08000924      0x0000000e    Code    RO      5270      i.__scatterload_zeroinit     mc_w.l(handlers.o)
27 0x08000932      0x00000022    Code    RO      4974      i.__printf_core              mc_w.l(printfb.o)
28 0x08000954      0x00000024    Code    RO      4879      i.fputc                      bsp_debug_usart.o
29 0x08000978      0x000000f8    Code    RO      4765      i.main                       main.o
30 0x08000a70      0x00000020    Data    RO      5266      Region$$Table               anon$$obj.o
```

这一段是某工程的 ROM 存储器分布映像，在 STM32 芯片中，ROM 区域的内容就是指存储到内部 FLASH 的代码。

1. 程序 ROM 的加载与执行空间

上述说明中有两段分别以“Load Region LR_ROM1”及“Execution Region ER_IROM1”开头的内容，它们分别描述程序的加载及执行空间。在芯片刚上电运行时，会加载程序及数据，例如它会从程序的存储区域加载到程序的执行区域，还把一些已初始化的全局变量

从 ROM 复制到 RAM 空间，以便程序运行时可以修改变量的内容。加载完成后，程序开始从执行区域开始执行。

在上面 map 文件的描述中，我们了解到加载及执行空间的基地址(Base)都是 0x08000000，它正好是 STM32 内部 FLASH 的首地址，即 STM32 的程序存储空间就直接是执行空间；它们的大小(Size)分别为 0x00000aa4 及 0x00000a90，执行空间的 ROM 比较小的原因就是部分 RW-data 类型的变量被拷贝到 RAM 空间了；它们的最大空间(Max)均为 0x00100000，即 1M 字节，它指的是内部 FLASH 的最大空间。

计算程序占用的空间时，需要使用加载区域的大小进行计算，本例子中应用程序使用的内部 FLASH 是从 0x08000000 至(0x08000000+0x00000aa4)地址的空间区域。

2. ROM 空间分布表

在加载及执行空间总体描述之后，紧接着一个 ROM 详细地址分布表，它列出了工程中的各个段(如函数、常量数据)所在的地址 Base Addr 及占用的空间 Size，列表中的 Type 说明了该段的类型，CODE 表示代码，DATA 表示数据，而 PAD 表示段之间的填充区域，它是无效的内容，PAD 区域往往是为了解决地址对齐的问题。

观察表中的最后一项，它的基地址是 0x08000a70，大小为 0x00000020，可知它占用的最高的地址空间为 0x08000a90，跟执行区域的最高地址 0x00000a90 一样，但它们比加载区域说明中的最高地址 0x8000aa4 要小，所以我们以加载区域的大小为准。对比表 50-1 的内部 FLASH 扇区地址分布表，可知仅使用扇区 0 就可以完全存储本应用程序，所以从扇区 1(地址 0x08004000)后的存储空间都可以作其它用途，使用这些存储空间时不会篡改应用程序空间的数据。

50.4 操作内部 FLASH 的库函数

为简化编程，STM32 标准库提供了一些库函数，它们封装了对内部 FLASH 写入数据操作寄存器的过程。

1. FLASH 解锁、上锁函数

对内部 FLASH 解锁、上锁的函数见代码清单 50-2。

代码清单 50-2 FLASH 解锁、上锁

```
1
2 #define FLASH_KEY1 ((uint32_t)0x45670123)
3 #define FLASH_KEY2 ((uint32_t)0xCDEF89AB)
4 /**
5  * @brief Unlocks the FLASH control register access
6  * @param None
7  * @retval None
8  */
9 void FLASH_Unlock(void)
10 {
11     if ((FLASH->CR & FLASH_CR_LOCK) != RESET) {
12         /* Authorize the FLASH Registers access */
13         FLASH->KEYR = FLASH_KEY1;
14         FLASH->KEYR = FLASH_KEY2;
15     }
16 }
```

```
17
18 /**
19  * @brief Locks the FLASH control register access
20  * @param None
21  * @retval None
22  */
23 void FLASH_Lock(void)
24 {
25     /* Set the LOCK Bit to lock the FLASH Registers access */
26     FLASH->CR |= FLASH_CR_LOCK;
27 }
```

解锁的时候，它对 FLASH_KEYR 寄存器写入两个解锁参数，上锁的时候，对 FLASH_CR 寄存器的 FLASH_CR_LOCK 位置 1。

2. 设置操作位数及擦除扇区

解锁后擦除扇区时可调用 FLASH_EraseSector 完成，见代码清单 50-3。

代码清单 50-3 擦除扇区

```
1 /**
2  * @brief Erases a specified FLASH Sector.
3  *
4  * @note If an erase and a program operations are requested simultaneously,
5  *       the erase operation is performed before the program one.
6  *
7  * @param FLASH_Sector: The Sector number to be erased.
8  *
9  * @note For STM32F42xxx/43xxx devices this parameter can be a value between
10 *       FLASH Sector 0 and FLASH Sector 23.
11 *
12 * @param VoltageRange: The device voltage range which defines the erase parallelism.
13 *       This parameter can be one of the following values:
14 *       @arg VoltageRange_1: when the device voltage range is 1.8V to 2.1V,
15 *       the operation will be done by byte (8-bit)
16 *       @arg VoltageRange_2: when the device voltage range is 2.1V to 2.7V,
17 *       the operation will be done by half word (16-bit)
18 *       @arg VoltageRange_3: when the device voltage range is 2.7V to 3.6V,
19 *       the operation will be done by word (32-bit)
20 *       @arg VoltageRange_4: when the device voltage range is 2.7V to 3.6V + External Vpp,
21 *       the operation will be done by double word (64-bit)
22 *
23 * @retval FLASH Status: The returned value can be: FLASH_BUSY, FLASH_ERROR_PROGRAM,
24 *       FLASH_ERROR_WRP, FLASH_ERROR_OPERATION or FLASH_COMPLETE.
25  */
26 FLASH_Status FLASH_EraseSector(uint32_t FLASH_Sector, uint8_t VoltageRange)
27 {
28     uint32_t tmp_psize = 0x0;
29     FLASH_Status status = FLASH_COMPLETE;
30
31     /* Check the parameters */
32     assert_param(IS_FLASH_SECTOR(FLASH_Sector));
33     assert_param(IS_VOLTAGERANGE(VoltageRange));
34
35     if (VoltageRange == VoltageRange_1) {
36         tmp_psize = FLASH_PSIZE_BYTE;
37     } else if (VoltageRange == VoltageRange_2) {
38         tmp_psize = FLASH_PSIZE_HALF_WORD;
39     } else if (VoltageRange == VoltageRange_3) {
40         tmp_psize = FLASH_PSIZE_WORD;
41     } else {
42         tmp_psize = FLASH_PSIZE_DOUBLE_WORD;
43     }
44     /* Wait for last operation to be completed */
45     status = FLASH_WaitForLastOperation();
46
47     if (status == FLASH_COMPLETE) {
48         /* if the previous operation is completed, proceed to erase the sector */
49     }
```

```
49     FLASH->CR &= CR_PSIZE_MASK;
50     FLASH->CR |= tmp_psize;
51     FLASH->CR &= SECTOR_MASK;
52     FLASH->CR |= FLASH_CR_SER | FLASH_Sector;
53     FLASH->CR |= FLASH_CR_STRT;
54
55     /* Wait for last operation to be completed */
56     status = FLASH_WaitForLastOperation();
57
58     /* if the erase operation is completed, disable the SER Bit */
59     FLASH->CR &= (~FLASH_CR_SER);
60     FLASH->CR &= SECTOR_MASK;
61 }
62 /* Return the Erase Status */
63 return status;
64 }
```

本函数包含两个输入参数，分别是要擦除的扇区号和工作电压范围，选择不同电压时实质是选择不同的数据操作位数，参数中可输入的宏在注释里已经给出。函数根据输入参数配置 PSIZE 位，然后擦除扇区，擦除扇区的时候需要等待一段时间，它使用 FLASH_WaitForLastOperation 等待，擦除完成的时候才会退出 FLASH_EraseSector 函数。

3. 写入数据

对内部 FLASH 写入数据不像对外部 SRAM 操作那样直接指针操作就完成了，还要设置一系列的寄存器，利用 FLASH_ProgramWord、FLASH_ProgramHalfWord 和 FLASH_ProgramByte 函数可按字、半字及字节单位写入数据，见代码清单 50-4。

代码清单 50-4 写入数据

```
1
2 /**
3  * @brief  Programs a word (32-bit) at a specified address.
4  *
5  * @note   This function must be used when the device voltage range is from 2.7V to 3.6V.
6  *
7  * @note   If an erase and a program operations are requested simultaneously,
8  *          the erase operation is performed before the program one.
9  *
10 * @param  Address: specifies the address to be programmed.
11 *          This parameter can be any address in Program memory zone or in OTP zone.
12 * @param  Data: specifies the data to be programmed.
13 * @retval FLASH Status: The returned value can be: FLASH_BUSY, FLASH_ERROR_PROGRAM,
14 *          FLASH_ERROR_WRP, FLASH_ERROR_OPERATION or FLASH_COMPLETE.
15 */
16 FLASH_Status FLASH_ProgramWord(uint32_t Address, uint32_t Data)
17 {
18     FLASH_Status status = FLASH_COMPLETE;
19
20     /* Check the parameters */
21     assert_param(IS_FLASH_ADDRESS(Address));
22
23     /* Wait for last operation to be completed */
24     status = FLASH_WaitForLastOperation();
25
26     if (status == FLASH_COMPLETE) {
27 /* if the previous operation is completed, proceed to program the new data */
28         FLASH->CR &= CR_PSIZE_MASK;
29         FLASH->CR |= FLASH_PSIZE_WORD;
30         FLASH->CR |= FLASH_CR_PG;
31
32         *(__IO uint32_t*)Address = Data;
33
34         /* Wait for last operation to be completed */
35         status = FLASH_WaitForLastOperation();
```



```
36
37     /* if the program operation is completed, disable the PG Bit */
38     FLASH->CR &= (~FLASH_CR_PG);
39 }
40 /* Return the Program Status */
41 return status;
42 }
```

看函数代码可了解到，使用指针进行赋值操作前设置了数据操作宽度，并设置了 PG 寄存器位，在赋值操作后，调用了 FLASH_WaitForLastOperation 函数等待写操作完毕。HalfWord 和 Byte 操作宽度的函数执行过程类似。

50.5 实验：读写内部 FLASH

在本小节中我们以实例讲解如何使用内部 FLASH 存储数据。

50.5.1 硬件设计

本实验仅操作了 STM32 芯片内部的 FLASH 空间，无需额外的硬件。

50.5.2 软件设计

本小节讲解的是“内部 FLASH 编程”实验，请打开配套的代码工程阅读理解。为了方便展示及移植，我们把操作内部 FLASH 相关的代码都编写到“bsp_internalFlash.c”及“bsp_internalFlash.h”文件中，这些文件是我们自己编写的，不属于标准库的内容，可根据您的喜好命名文件。

1. 程序设计要点

- (1) 对内部 FLASH 解锁；
- (2) 找出空闲扇区，擦除目标扇区；
- (3) 进行读写测试。

2. 代码分析

硬件定义

读写内部 FLASH 不需要用到任何外部硬件，不过在擦写时常常需要知道各个扇区的基地址，我们把这些基地址定义到 bsp_internalFlash.h 文件中，见代码清单 52-1。

代码清单 50-5 各个扇区的基地址(bsp_internalFlash.h 文件)

```
1
2 /* 各个扇区的基地址 */
3 #define ADDR_FLASH_SECTOR_0 ((uint32_t)0x08000000)
4 #define ADDR_FLASH_SECTOR_1 ((uint32_t)0x08004000)
5 #define ADDR_FLASH_SECTOR_2 ((uint32_t)0x08008000)
6 #define ADDR_FLASH_SECTOR_3 ((uint32_t)0x0800C000)
7 #define ADDR_FLASH_SECTOR_4 ((uint32_t)0x08010000)
8 #define ADDR_FLASH_SECTOR_5 ((uint32_t)0x08020000)
9 #define ADDR_FLASH_SECTOR_6 ((uint32_t)0x08040000)
10 #define ADDR_FLASH_SECTOR_7 ((uint32_t)0x08060000)
11 #define ADDR_FLASH_SECTOR_8 ((uint32_t)0x08080000)
12 #define ADDR_FLASH_SECTOR_9 ((uint32_t)0x080A0000)
13 #define ADDR_FLASH_SECTOR_10 ((uint32_t)0x080C0000)
```

```
14 #define ADDR_FLASH_SECTOR_11      ((uint32_t)0x080E0000)
15
16 #define ADDR_FLASH_SECTOR_12      ((uint32_t)0x08100000)
17 #define ADDR_FLASH_SECTOR_13      ((uint32_t)0x08104000)
18 #define ADDR_FLASH_SECTOR_14      ((uint32_t)0x08108000)
19 #define ADDR_FLASH_SECTOR_15      ((uint32_t)0x0810C000)
20 #define ADDR_FLASH_SECTOR_16      ((uint32_t)0x08110000)
21 #define ADDR_FLASH_SECTOR_17      ((uint32_t)0x08120000)
22 #define ADDR_FLASH_SECTOR_18      ((uint32_t)0x08140000)
23 #define ADDR_FLASH_SECTOR_19      ((uint32_t)0x08160000)
24 #define ADDR_FLASH_SECTOR_20      ((uint32_t)0x08180000)
25 #define ADDR_FLASH_SECTOR_21      ((uint32_t)0x081A0000)
26 #define ADDR_FLASH_SECTOR_22      ((uint32_t)0x081C0000)
27 #define ADDR_FLASH_SECTOR_23      ((uint32_t)0x081E0000)
```

这些宏跟表 50-1 中的地址说明一致。

根据扇区地址计算 SNB 寄存器的值

在擦除操作时，需要向 FLASH 控制寄存器 FLASH_CR 的 SNB 位写入要擦除的扇区号，固件库把各个扇区对应的寄存器值使用宏定义到了 stm32f4xx_flash.h 文件。为了便于使用，我们自定义了一个 GetSector 函数，根据输入的内部 FLASH 地址，找出其所在的扇区，并返回该扇区对应的 SNB 位寄存器值，见代码清单 52-2。

代码清单 50-6 写入到 SNB 寄存器位的值（stm32f4xx_flash.h 及 bsp_internalFlash.c 文件）

```
1 /*固件库定义的用于扇区写入到 SNB 寄存器位的宏(stm32f4xx_flash.h 文件)*/
2 #define FLASH_Sector_0      ((uint16_t)0x0000)
3 #define FLASH_Sector_1      ((uint16_t)0x0008)
4 #define FLASH_Sector_2      ((uint16_t)0x0010)
5 #define FLASH_Sector_3      ((uint16_t)0x0018)
6 #define FLASH_Sector_4      ((uint16_t)0x0020)
7 #define FLASH_Sector_5      ((uint16_t)0x0028)
8 #define FLASH_Sector_6      ((uint16_t)0x0030)
9 #define FLASH_Sector_7      ((uint16_t)0x0038)
10 #define FLASH_Sector_8      ((uint16_t)0x0040)
11 #define FLASH_Sector_9      ((uint16_t)0x0048)
12 #define FLASH_Sector_10     ((uint16_t)0x0050)
13 #define FLASH_Sector_11     ((uint16_t)0x0058)
14 #define FLASH_Sector_12     ((uint16_t)0x0080)
15 #define FLASH_Sector_13     ((uint16_t)0x0088)
16 #define FLASH_Sector_14     ((uint16_t)0x0090)
17 #define FLASH_Sector_15     ((uint16_t)0x0098)
18 #define FLASH_Sector_16     ((uint16_t)0x00A0)
19 #define FLASH_Sector_17     ((uint16_t)0x00A8)
20 #define FLASH_Sector_18     ((uint16_t)0x00B0)
21 #define FLASH_Sector_19     ((uint16_t)0x00B8)
22 #define FLASH_Sector_20     ((uint16_t)0x00C0)
23 #define FLASH_Sector_21     ((uint16_t)0x00C8)
24 #define FLASH_Sector_22     ((uint16_t)0x00D0)
25 #define FLASH_Sector_23     ((uint16_t)0x00D8)
26
27 /*定义在 bsp_internalFlash.c 文件中的函数*/
28 /**
29  * @brief  根据输入的地址给出它所在的 sector
30  *          例如：
31  *          uwStartSector = GetSector(FLASH_USER_START_ADDR);
32  *          uwEndSector = GetSector(FLASH_USER_END_ADDR);
33  * @param  Address: 地址
34  * @retval 地址所在的 sector
35  */
36 static uint32_t GetSector(uint32_t Address)
37 {
38     uint32_t sector = 0;
39 }
```

```
40 if ((Address < ADDR_FLASH_SECTOR_1) && (Address >= ADDR_FLASH_SECTOR_0)) {  
41     sector = FLASH_Sector_0;  
42 } else if ((Address < ADDR_FLASH_SECTOR_2) && (Address >= ADDR_FLASH_SECTOR_1)) {  
43     sector = FLASH_Sector_1;  
44 }  
45  
46 /*此处省略扇区 2-扇区 21 的内容*/  
47  
48 else if ((Address < ADDR_FLASH_SECTOR_23) && (Address >= ADDR_FLASH_SECTOR_22)) {  
49     sector = FLASH_Sector_22;  
50 } else { /*(Address < FLASH_END_ADDR) && (Address >= ADDR_FLASH_SECTOR_23)*/  
51     sector = FLASH_Sector_23;  
52 }  
53 return sector;  
54 }
```

代码中固件库定义的宏 FLASH_Sector_0-23 对应的值是跟寄存器说明一致的，见图 50-3。

Bits 7:3	SNB: Sector number
These bits select the sector to erase.	
0000:	sector 0
0001:	sector 1
...	
01011:	sector 11
01100:	not allowed
01101:	not allowed
01110:	not allowed
01111:	not allowed
10000:	sector 12
10001:	sector 13
...	
11011:	sector 23
11100:	not allowed
11101:	not allowed
11110:	not allowed
11111:	not allowed

图 50-3 FLASH_CR 寄存器的 SNB 位的值

GetSector 函数根据输入的地址与各个扇区的基地址进行比较，找出它所在的扇区，并使用固件库中的宏，返回扇区对应的 SNB 值。

读写内部 FLASH

一切准备就绪，可以开始对内部 FLASH 进行擦写，这个过程不需要初始化任何外设，只要按解锁、擦除及写入的流程走就可以了，见代码清单 52-3。

代码清单 50-7 对内部地 FLASH 进行读写测试(bsp_internalFlash.c 文件)

```
1  
2 /*准备写入的测试数据*/  
3 #define DATA_32 ((uint32_t)0x00000000)  
4 /* 要擦除内部 FLASH 的起始地址 */  
5 #define FLASH_USER_START_ADDR ADDR_FLASH_SECTOR_8  
6 /* 要擦除内部 FLASH 的结束地址 */  
7 #define FLASH_USER_END_ADDR ADDR_FLASH_SECTOR_12  
8  
9 /**  
10  * @brief InternalFlash_Test, 对内部 FLASH 进行读写测试  
11  * @param None  
12  * @retval None  
13  */  
14 int InternalFlash_Test(void)  
15 {  
16     /*要擦除的起始扇区(包含)及结束扇区(不包含)，如 8-12，表示擦除 8、9、10、11 扇区*/  
17     uint32_t uwStartSector = 0;  
18     uint32_t uwEndSector = 0;  
19 }
```

```
20     uint32_t uwAddress = 0;
21     uint32_t uwSectorCounter = 0;
22
23     __IO uint32_t uwData32 = 0;
24     __IO uint32_t uwMemoryProgramStatus = 0;
25
26     /* FLASH 解锁 ***** */
27     /* 使能访问 FLASH 控制寄存器 */
28     FLASH_Unlock();
29
30     /* 擦除用户区域 (用户区域指程序本身没有使用的空间, 可以自定义) */
31     /* 清除各种 FLASH 的标志位 */
32     FLASH_ClearFlag(FLASH_FLAG_EOP | FLASH_FLAG_OPERR | FLASH_FLAG_WRPERR |
33                     FLASH_FLAG_PGERR | FLASH_FLAG_PGPERR | FLASH_FLAG_PGSERR);
34
35
36     uwStartSector = GetSector(FLASH_USER_START_ADDR);
37     uwEndSector = GetSector(FLASH_USER_END_ADDR);
38
39     /* 开始擦除操作 */
40     uwSectorCounter = uwStartSector;
41     while (uwSectorCounter <= uwEndSector) {
42         /* VoltageRange_3 以“字”的大小进行操作 */
43         if (FLASH_EraseSector(uwSectorCounter, VoltageRange_3) != FLASH_COMPLETE)
44         {
45             /* 擦除出错, 返回, 实际应用中可加入处理 */
46             return -1;
47         }
48         /* 计数器指向下一个扇区 */
49         if (uwSectorCounter == FLASH_Sector_11) {
50             uwSectorCounter += 40;
51         } else {
52             uwSectorCounter += 8;
53         }
54     }
55
56     /* 以“字”的大小为单位写入数据 ***** */
57     uwAddress = FLASH_USER_START_ADDR;
58
59     while (uwAddress < FLASH_USER_END_ADDR) {
60         if (FLASH_ProgramWord(uwAddress, DATA_32) == FLASH_COMPLETE) {
61             uwAddress = uwAddress + 4;
62         } else {
63             /* 写入出错, 返回, 实际应用中可加入处理 */
64             return -1;
65         }
66     }
67
68     /* 给 FLASH 上锁, 防止内容被篡改 */
69     FLASH_Lock();
70
71
72     /* 从 FLASH 中读取数据进行校验 ***** */
73     /* MemoryProgramStatus = 0: 写入的数据正确
74        MemoryProgramStatus != 0: 写入的数据错误, 其值为错误的个数 */
75     uwAddress = FLASH_USER_START_ADDR;
76     uwMemoryProgramStatus = 0;
77
78     while (uwAddress < FLASH_USER_END_ADDR) {
79         uwData32 = *(__IO uint32_t*)uwAddress;
80
81         if (uwData32 != DATA_32) {
82             uwMemoryProgramStatus++;
83         }
84     }
```

```
85     uwAddress = uwAddress + 4;
86 }
87 /* 数据校验不正确 */
88 if (uwMemoryProgramStatus) {
89     return -1;
90 } else { /*数据校验正确*/
91     return 0;
92 }
93 }
94
```

该函数的执行过程如下：

- (1) 调用 FLASH_Unlock 解锁；
- (2) 调用 FLASH_ClearFlag 清除各种标志位；
- (3) 调用 GetSector 根据起始地址及结束地址计算要擦除的扇区；
- (4) 调用 FLASH_EraseSector 擦除扇区，擦除时按字为单位进行操作；
- (5) 调用 FLASH_ProgramWord 函数向起始地址至结束地址的存储区域都写入数值“DATA_32”；
- (6) 调用 FLASH_Lock 上锁；
- (7) 使用指针读取数据内容并校验。

main 函数

最后我们来看看 main 函数的执行流程，见代码清单 52-4。

代码清单 50-8 main 函数(main.c 文件)

```
1  /**
2   * @brief 主函数
3   * @param 无
4   * @retval 无
5   */
6  int main(void)
7  {
8      /*初始化 USART，配置模式为 115200 8-N-1*/
9      Debug_USART_Config();
10     LED_GPIO_Config();
11
12     LED_BLUE;
13     /*调用 printf 函数，因为重定向了 fputc，printf 的内容会输出到串口*/
14     printf("this is a usart printf demo. \r\n");
15     printf("\r\n 欢迎使用野火 STM32 F407 开发板.\r\n");
16     printf("正在进行读写内部 FLASH 实验，请耐心等待\r\n");
17
18     if (InternalFlash_Test()==0) {
19         LED_GREEN;
20         printf("读写内部 FLASH 测试成功\r\n");
21     } else {
22         printf("读写内部 FLASH 测试失败\r\n");
23         LED_RED;
24     }
25 }
26
```

main 函数中初始化了用于指示调试信息的 LED 及串口后，直接调用了 InternalFlash_Test 函数，进行读写测试并根据测试结果输出调试信息。

50.5.3 下载验证

用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。在串口调试助手可看到擦写内部 FLASH 的调试信息。

